

재밍 환경에서 Speech-to-Speech JSCC를 위한 권장 아키텍처, 채널 적응적 레이어 사용 및 재밍 적응 방법

Speech JSCC 설계 초안

June 29, 2026

전제 및 설계 방향

본 문서는 무선 재밍 환경에서 speech-to-speech JSCC를 구현하기 위한 1차 설계안이다. 여기서는 ASR/TTS, KG 기반 symbolic semantic side channel, 디지털 FEC 기반 RVQ index 전송을 주력 구조로 사용하지 않는다. 기본 방향은 다음과 같다.

Speech waveform $\rightarrow$ neural speech codec representation $\rightarrow$ JSCC encoder $\rightarrow$ wireless channel with jamming $\rightarrow$ JSCC decoder $\rightarrow$ codec decoder $\rightarrow$ speech waveform

즉, 목표는 RVQ codebook index를 bit-perfect하게 복원하는 것이 아니라, 재밍과 Rayleigh fading으로 왜곡된 수신 신호로부터 speech representation을 복원하여 최종 waveform의 명료도, 음질, 화자성, 작전 키워드 전달률을 유지하는 것이다.

1 5. 권장 아키텍처

1.1 5.1 전체 송수신 구조

송신 음성을 $\mathbf{s} \in \mathbb{R}^{T_s}$ 라고 하자. 사전 학습된 speech codec encoder를 E_ψ , decoder를 D_ψ 라고 둔다. 초기 구현에서는 E_ψ 와 D_ψ 를 freeze하고, JSCC encoder/decoder만 학습한다.

SpeechTokenizer, EnCodec, Mimi류 codec은 음성을 연속 latent 또는 RVQ 기반 codebook representation으로 변환한다. RVQ layer 수를 L 이라고 하면, 시간 index t 와 RVQ layer ℓ 에 대해

$$q_{\ell,t} \in \{1, 2, \dots, K_\ell\}, \quad \ell = 1, \dots, L, \quad t = 1, \dots, T \quad (1)$$

와 같은 codebook index를 얻을 수 있다. 각 codebook을 $\mathcal{C}_\ell = \{\mathbf{c}_{\ell,1}, \dots, \mathbf{c}_{\ell,K_\ell}\}$ 라고 하면, index에 대응하는 embedding은

$$\mathbf{e}_{\ell,t} = \mathbf{c}_{\ell,q_{\ell,t}} \in \mathbb{R}^{d_\ell} \quad (2)$$

이다. 본 JSCC 시스템에서는 $q_{\ell,t}$ 를 binary index로 변환하여 전송하는 것이 아니라, layer 별 embedding sequence

$$\mathbf{E} = \{\mathbf{e}_{\ell,t} : \ell = 1, \dots, L, t = 1, \dots, T\} \quad (3)$$

또는 pre-quantization latent를 JSCC encoder 입력으로 사용한다.

JSCC encoder f_θ 는 speech codec representation과 채널 상태 벡터 $\mathbf{c}$ 를 입력받아 complex channel symbols $\mathbf{x} \in \mathbb{C}^M$ 를 생성한다.

$$\mathbf{x} = f_\theta(\mathbf{E}, \mathbf{c}), \quad \frac{1}{M} \mathbb{E} [\|\mathbf{x}\|_2^2] \leq P_s. \quad (4)$$

여기서 $\mathbf{c}$ 는 추정 SNR/SINR, CSI 품질, jammer-to-signal ratio(JSR), 재밍 mask 통계 등을 포함한다.

수신기는 채널 출력 $\mathbf{y}$ 와 추정 채널 상태 $\hat{\mathbf{c}}$ 를 입력으로 받아 reconstructed codec representation을 생성한다.

$$\hat{\mathbf{E}} = g_\phi(\mathbf{y}, \hat{\mathbf{c}}). \quad (5)$$

그 후 codec decoder가 waveform을 복원한다.

$$\hat{\mathbf{s}} = D_\psi(\hat{\mathbf{E}}). \quad (6)$$

1.2 5.2 Rayleigh fading 및 재밍 채널

1차 구현에서는 단순 Rayleigh fading을 사용한다. flat fading 모델은

$$\mathbf{y} = h\mathbf{x} + g\mathbf{j} + \mathbf{n} \quad (7)$$

로 둔다. 여기서

$$h \sim \mathcal{CN}(0, 1), \quad g \sim \mathcal{CN}(0, 1), \quad \mathbf{n} \sim \mathcal{CN}(\mathbf{0}, N_0 \mathbf{I}) \quad (8)$$

이고 $\mathbf{j}$ 는 jammer signal이다. 송신 전력과 재밍 전력을 각각 P_s 와 P_j 라고 하면,

$$\text{JSR} = \frac{P_j}{P_s}, \quad \gamma_{\text{eff}} = \frac{|h|^2 P_s}{|g|^2 P_j + N_0} \quad (9)$$

로 effective SINR을 정의한다.

OFDM 확장 모델에서는 subcarrier k 와 OFDM symbol index m 에 대해

$$Y_{k,m} = H_{k,m}X_{k,m} + G_{k,m}J_{k,m} + N_{k,m} \quad (10)$$

를 사용한다. 이 경우 재밍은 시간-주파수 mask Ω_J 로 표현할 수 있다.

$$J_{k,m} = \mathbb{I}\{(k, m) \in \Omega_J\} \sqrt{P_{j,k,m}} U_{k,m}, \quad U_{k,m} \sim \mathcal{CN}(0, 1). \quad (11)$$

1.3 5.3 Soft codebook projection

JSCC decoder가 hard index를 직접 출력하도록 강제하면, 재밍 상황에서 작은 왜곡이 큰 index 오류로 바뀔 수 있다. 따라서 수신기는 layer 별 token distribution 또는 continuous embedding을 출력하도록 설계한다.

JSCC decoder 출력에서 layer ℓ , time t 의 logit을 $\mathbf{a}_{\ell,t} \in \mathbb{R}^{K_\ell}$ 라고 하면,

$$\pi_{\ell,t,k} = \frac{\exp(a_{\ell,t,k}/\tau)}{\sum_{r=1}^{K_\ell} \exp(a_{\ell,t,r}/\tau)} \quad (12)$$

이고, soft codebook projection은

$$\tilde{\mathbf{e}}_{\ell,t} = \sum_{k=1}^{K_\ell} \pi_{\ell,t,k} \mathbf{c}_{\ell,k} \quad (13)$$

이다. 학습 중에는 soft projection을 사용하고, 평가 시에는 필요에 따라

$$\hat{q}_{\ell,t} = \arg \max_k \pi_{\ell,t,k} \quad (14)$$

를 사용해 hard projection도 비교한다.

1.4 5.4 Latent refinement module

재밍이 강하면 g_ϕ 가 출력한 $\hat{\mathbf{E}}$ 에 잔류 왜곡이 남는다. 이를 줄이기 위해 optional refinement network R_ω 를 둔다.

$$\tilde{\mathbf{E}} = R_\omega(\hat{\mathbf{E}}, \hat{\mathbf{c}}), \quad \hat{\mathbf{s}} = D_\psi(\tilde{\mathbf{E}}). \quad (15)$$

R_ω 는 temporal context와 RVQ layer context를 모두 보는 작은 Transformer, Conformer, 또는 temporal convolution network로 구현할 수 있다. 1차 구현에서는 R_ω 를 제외한 baseline을 먼저 만들고, 이후 ablation으로 추가한다.

1.5 5.5 학습 목적함수

전체 학습 loss는 codec latent reconstruction, waveform/perceptual reconstruction, speaker similarity, intelligibility 관련 loss를 결합한다.

$$\begin{aligned}\mathcal{L}_{\text{total}} = & \lambda_{\text{lat}}\mathcal{L}_{\text{lat}} + \lambda_{\text{tok}}\mathcal{L}_{\text{tok}} + \lambda_{\text{stft}}\mathcal{L}_{\text{STFT}} \\ & + \lambda_{\text{spk}}\mathcal{L}_{\text{spk}} + \lambda_{\text{int}}\mathcal{L}_{\text{int}} + \lambda_{\text{pow}}\mathcal{L}_{\text{power}}.\end{aligned}\quad (16)$$

기본 latent loss는

$$\mathcal{L}_{\text{lat}} = \sum_{\ell=1}^L w_{\ell} \sum_{t=1}^T \|e_{\ell,t} - \hat{e}_{\ell,t}\|_2^2 \quad (17)$$

이다. 여기서 w_{ℓ} 은 layer 중요도이다. token CE loss를 사용할 경우

$$\mathcal{L}_{\text{tok}} = - \sum_{\ell=1}^L w_{\ell} \sum_{t=1}^T \log \pi_{\ell,t,q_{\ell,t}} \quad (18)$$

로 둔다. 전력 제약은

$$\mathcal{L}_{\text{power}} = \left[\frac{1}{M} \|\mathbf{x}\|_2^2 - P_s \right]_+^2 \quad (19)$$

로 둘 수 있다.

1.6 5.6 단계적 학습 전략

- 1) **Codec freeze 단계**: E_{ψ}, D_{ψ} 를 고정하고 f_{θ}, g_{ϕ} 만 학습한다.
- 2) **Latent-only 단계**: waveform decoder까지 연결하지 않고 $\mathcal{L}_{\text{lat}}$ 와 $\mathcal{L}_{\text{tok}}$ 만으로 학습한다.
- 3) **Waveform 연결 단계**: D_{ψ} 를 연결하여 STFT/mel loss와 perceptual metric을 추가한다.
- 4) **Jamming curriculum 단계**: AWGN $\rightarrow$ Rayleigh $\rightarrow$ barrage jamming $\rightarrow$ narrowband/burst jamming 순서로 channel distribution을 넓힌다.
- 5) **Refinement 추가 단계**: baseline 수렴 후 R_w 를 추가하고 강한 JSR 구간에서 성능을 비교한다.

2 6. 채널 적응적 레이어 사용

2.1 6.1 의미 정리

여기서 “레이어 사용”은 neural network layer를 송신한다는 뜻이 아니다. 송수신기는 pretrained speech codec encoder/decoder와 codebook을 이미 공유하고 있다. 채널 적응적 레이어 사용이란 RVQ layer별 codec representation $\mathbf{E}_1, \dots, \mathbf{E}_L$ 에 대해, 채널 상태에 따라 JSCC encoder가 할당하는 표현력, channel symbol 수, 전력, gating weight를 바꾸는 것을 의미한다.

2.2 6.2 Layer-wise gating

채널 상태 벡터를

$$\mathbf{c} = [\hat{\gamma}_{\text{eff}}, \widehat{\text{JSR}}, \text{NMSE}_{\text{CSI}}, p_{\text{jam}}, \rho_{\text{mask}}]^{\top} \quad (20)$$

라고 하자. p_{jam} 은 jammer type posterior, ρ_{mask} 는 jammed resource ratio이다. Layer gate는

$$\alpha = \sigma(\text{MLP}_{\eta}(\mathbf{c})), \quad \alpha_{\ell} \in [0, 1] \quad (21)$$

로 정의한다. JSCC encoder 입력은

$$\mathbf{E}'_\ell = \alpha_\ell \mathbf{E}_\ell, \quad \ell = 1, \dots, L \quad (22)$$

로 조정된다. 채널이 좋으면 $\alpha_\ell \approx 1$ 이 모든 layer에 대해 유지되고, 채널이 나쁘면 content-sensitive layer의 α_ℓ 만 크게 유지된다.

2.3 6.3 Layer-wise channel-use allocation

전체 channel symbol budget을 M 이라고 하자. Layer ℓ 에 할당되는 channel use 수를 M_ℓ 로 두면,

$$\sum_{\ell=1}^L M_\ell \leq M, \quad M_\ell \geq 0. \quad (23)$$

채널 상태 기반 allocation은

$$\beta_\ell = \frac{\exp(u_\ell)}{\sum_{r=1}^L \exp(u_r)}, \quad \mathbf{u} = \text{MLP}_\xi(\mathbf{c}) \quad (24)$$

로 계산하고,

$$M_\ell = \lfloor M\beta_\ell \rfloor \quad (25)$$

로 둘 수 있다. 단, content-sensitive layer에는 최소 channel use를 보장한다.

$$M_\ell \geq M_\ell^{\min}, \quad \ell \in \mathcal{B} \quad (26)$$

여기서 $\mathcal{B}$ 는 base representation layer 집합이다.

2.4 6.4 Layer-wise power allocation

동일한 channel use를 쓰더라도 layer 별 power를 다르게 줄 수 있다.

$$\sum_{\ell=1}^L P_\ell \leq P_{\text{tot}}, \quad P_\ell = P_{\text{tot}} \frac{\exp(v_\ell)}{\sum_{r=1}^L \exp(v_r)}, \quad \mathbf{v} = \text{MLP}_\zeta(\mathbf{c}). \quad (27)$$

layer ℓ 의 JSCC symbol을 $\mathbf{x}_\ell$ 이라고 하면 최종 송신 symbol은

$$\mathbf{x} = \sum_{\ell=1}^L \sqrt{P_\ell} \frac{\mathbf{x}_\ell}{\sqrt{\mathbb{E}\|\mathbf{x}_\ell\|_2^2}} \quad (28)$$

로 구성할 수 있다.

2.5 6.5 Rule-based 초기 정책

처음부터 gate와 allocation을 모두 학습하면 디버깅이 어렵다. 따라서 1차 구현에서는 threshold 기반 rule을 사용한다.

$$\mathcal{A}(\hat{\gamma}_{\text{eff}}) = \begin{cases} \{1, 2, \dots, L\}, & \hat{\gamma}_{\text{eff}} \geq \Gamma_3, \\ \{1, 2, \dots, L_2\}, & \Gamma_2 \leq \hat{\gamma}_{\text{eff}} < \Gamma_3, \\ \{1, 2, \dots, L_1\}, & \Gamma_1 \leq \hat{\gamma}_{\text{eff}} < \Gamma_2, \\ \mathcal{B}, & \hat{\gamma}_{\text{eff}} < \Gamma_1, \end{cases} \quad (29)$$

여기서 $\mathcal{A}$ 는 활성화되는 RVQ representation layer 집합이다. 예를 들어 $L = 8, L_2 = 6, L_1 = 4, \mathcal{B} = \{1, 2\}$ 로 둘 수 있다. 이 수치는 가정이므로 반드시 layer ablation으로 검증한다.

2.6 6.6 Layer ablation을 통한 중요도 추정

각 RVQ layer의 중요도는 실험적으로 추정한다. layer subset $\mathcal{S} \subseteq \{1, \dots, L\}$ 만 유지하고 나머지를 mask 또는 zero로 둔 후 waveform을 복원한다.

$$\hat{\mathbf{s}}_{\mathcal{S}} = D_{\psi}(\{\mathbf{E}_{\ell} : \ell \in \mathcal{S}\}, \{\mathbf{0} : \ell \notin \mathcal{S}\}). \quad (30)$$

layer ℓ 의 intelligibility sensitivity는

$$\Delta_{\ell}^{\text{WER}} = \text{WER}(\hat{\mathbf{s}}_{\setminus \ell}) - \text{WER}(\hat{\mathbf{s}}_{\text{all}}) \quad (31)$$

로 정의할 수 있다. speaker sensitivity는 speaker embedding function Φ_{spk} 를 사용해

$$\Delta_{\ell}^{\text{spk}} = \cos(\Phi_{\text{spk}}(\mathbf{s}), \Phi_{\text{spk}}(\hat{\mathbf{s}}_{\text{all}})) - \cos(\Phi_{\text{spk}}(\mathbf{s}), \Phi_{\text{spk}}(\hat{\mathbf{s}}_{\setminus \ell})) \quad (32)$$

로 계산한다. 최종 layer weight는

$$w_{\ell} = a\Delta_{\ell}^{\text{WER}} + b\Delta_{\ell}^{\text{STOI}} + c\Delta_{\ell}^{\text{spk}} \quad (33)$$

처럼 둘 수 있다.

3 7. 재밍 적응 방법

3.1 7.1 재밍 모델

1차 시뮬레이터는 다음 jammer를 지원한다.

Barrage jamming 전체 대역 또는 전체 symbol에 white Gaussian jammer를 더한다.

$$\mathbf{j} \sim \mathcal{CN}(\mathbf{0}, P_j \mathbf{I}). \quad (34)$$

Narrowband jamming OFDM resource 중 일부 subcarrier 집합 $\mathcal{K}_J$ 만 공격한다.

$$J_{k,m} = \mathbb{I}\{k \in \mathcal{K}_J\} \sqrt{P_j} U_{k,m}. \quad (35)$$

Burst jamming 특정 시간 구간 $\mathcal{T}_J$ 만 공격한다.

$$J_{k,m} = \mathbb{I}\{m \in \mathcal{T}_J\} \sqrt{P_j} U_{k,m}. \quad (36)$$

Pilot jamming pilot resource Ω_P 에 집중적으로 재밍을 가한다.

$$J_{k,m} = \mathbb{I}\{(k, m) \in \Omega_P\} \sqrt{P_j} U_{k,m}. \quad (37)$$

이는 CSI 추정 오차를 증가시켜 JSCC decoder 조건 정보 $\hat{\mathbf{c}}$ 를 악화시킨다.

3.2 7.2 Jamming-augmented training

재밍 적응의 가장 기본은 학습 시 다양한 jammer distribution을 포함하는 것이다. 학습 objective는

$$\min_{\theta, \phi, \omega} \mathbb{E}_{\mathbf{s} \sim \mathcal{D}} \mathbb{E}_{h, g, j, n \sim \mathcal{P}_{\text{ch}}} [\mathcal{L}_{\text{total}}(\mathbf{s}, \hat{\mathbf{s}})] \quad (38)$$

이다. 여기서 $\mathcal{P}_{\text{ch}}$ 는 SNR, JSR, Rayleigh fading, jammer type, jammer mask를 모두 randomize한다.

더 강한 robust training은 worst-case 항을 추가한다.

$$\min_{\theta, \phi, \omega} \mathbb{E}_{\mathbf{s}, h, n} \left[\mathcal{L}_{\text{nominal}} + \rho \max_{j \in \mathcal{J}(P_j, \Omega_J)} \mathcal{L}_{\text{total}}(\mathbf{s}, \hat{\mathbf{s}}) \right]. \quad (39)$$

실제 구현에서는 max-jammer를 완전히 풀기보다, 학습 batch마다 여러 jammer 후보를 샘플링하고 loss가 큰 case를 선택한다.

3.3 7.3 Channel-state conditioned decoding

JSCC decoder는 수신 신호만 보는 것이 아니라 채널 상태와 재밍 상태를 조건으로 받아야 한다.

$$\hat{\mathbf{E}} = g_\phi(\mathbf{y}, \hat{\mathbf{c}}), \quad \hat{\mathbf{c}} = [\hat{\gamma}_{\text{eff}}, \widehat{\text{JSR}}, \text{NMSE}_{\text{CSI}}, \hat{p}_{\text{jam}}, \hat{p}_{\text{mask}}]. \quad (40)$$

Pilot 기반 CSI 추정

$$\hat{H}_{k,m}^{\text{LS}} = \frac{Y_{k,m}}{X_{k,m}}, \quad (k, m) \in \Omega_P \quad (41)$$

로 시작하고, CSI 품질은

$$\text{NMSE}_{\text{CSI}} = \frac{\mathbb{E} [\|H - \hat{H}\|_2^2]}{\mathbb{E} [\|H\|_2^2]} \quad (42)$$

로 측정한다. 실제 수신기에서는 H 를 알 수 없으므로 validation에서는 true NMSE를 기록하고, online decoder 조건에는 residual error, pilot EVM, jammer energy estimate 등을 넣는다.

3.4 7.4 Jammer-aware layer allocation

재밍이 일부 시간-주파수 resource에 집중되면, content-sensitive layer representation을 상대적으로 깨끗한 resource에 할당한다. OFDM resource $r = (k, m)$ 의 신뢰도를

$$R_r = \frac{|\hat{H}_r|^2}{\hat{\sigma}_{J,r}^2 + \hat{\sigma}_N^2 + \epsilon} \quad (43)$$

로 두고, layer ℓ 의 중요도를 w_ℓ 라고 하자. resource assignment는 다음 문제로 쓸 수 있다.

$$\begin{aligned} \max_{a_{\ell,r}} \quad & \sum_{\ell=1}^L \sum_{r \in \mathcal{R}} w_\ell a_{\ell,r} \log(1 + R_r) \\ \text{s.t.} \quad & \sum_{\ell=1}^L a_{\ell,r} \leq 1, \quad a_{\ell,r} \in \{0, 1\}, \\ & \sum_{r \in \mathcal{R}} a_{\ell,r} = M_\ell. \end{aligned} \quad (44)$$

1차 구현에서는 이 최적화를 풀지 말고, R_r 를 기준으로 resource를 정렬한 뒤 중요 layer부터 배정하는 greedy policy를 사용한다.

3.5 7.5 재밍 적응형 latent denoising

재밍으로 인한 왜곡은 random noise와 달리 특정 resource 또는 시간 구간에 구조적으로 나타난다. 따라서 refinement network는 jammer mask estimate를 조건으로 사용한다.

$$\tilde{\mathbf{E}} = R_\omega(\hat{\mathbf{E}}, \hat{\mathbf{M}}_J, \hat{\mathbf{c}}) \quad (45)$$

여기서 $\hat{\mathbf{M}}_J \in [0, 1]^{K \times M}$ 는 resource별 재밍 가능성 map이다. denoising loss는 강한 재밍 구간의 layer reconstruction을 더 크게 가중한다.

$$\mathcal{L}_{\text{denoise}} = \sum_{\ell, t} (w_\ell + \mu \hat{m}_t) \|\mathbf{e}_{\ell, t} - \tilde{\mathbf{e}}_{\ell, t}\|_2^2. \quad (46)$$

3.6 7.6 평가 지표

최소한 다음 지표를 보고한다.

- 무선 계층: effective SINR, JSR, CSI NMSE, pilot EVM.
- codec/latent 계층: layer 별 embedding MSE, token top-1/top-5 accuracy, soft projection entropy.
- 음성 계층: STOI, PESQ 또는 ViSQOL, multi-scale STFT distance, SI-SDR.
- 화자성: speaker embedding cosine similarity.
- 작전 통신 명료성: keyword accuracy, command word accuracy, optional ASR-WER. ASR은 시스템 구성 요소가 아니라 평가 도구로만 사용한다.

4 Codex 개발 계획 및 지시사항

4.1 개발 원칙

- 1) 먼저 speech 모델 없이 synthetic latent tensor로 JSCC와 channel/jammer를 검증한다.
- 2) 그 다음 SpeechTokenizer 또는 EnCodec wrapper를 붙인다.
- 3) codec encoder/decoder는 freeze하고, JSCC encoder/decoder와 optional refiner만 학습한다.
- 4) 디지털 FEC/index 전송 구현으로 흐르지 않도록 한다. 본 프로젝트의 핵심은 continuous/soft codec representation JSCC이다.
- 5) 모든 실험은 config 파일로 재현 가능해야 한다.

4.2 권장 프로젝트 구조

```
speech-jscc/  
  configs/  
    base.yaml  
    channel_rayleigh.yaml  
    jammer.yaml  
    model_jscc.yaml  
    train.yaml  
  src/  
    codecs/  
      base_codec.py  
      speeche tokenizer_wrapper.py  
      encodec_wrapper.py  
    channels/  
      rayleigh.py  
      ofdm.py  
      jammer.py  
      pilot.py  
      metrics.py  
    models/  
      jscc_encoder.py  
      jscc_decoder.py  
      layer_gate.py  
      soft_codebook.py
```

```

    latent_refiner.py
losses/
    latent_losses.py
    spectral_losses.py
    speaker_losses.py
data/
    speech_dataset.py
    collate.py
train/
    train_latent_jscc.py
    train_waveform_jscc.py
eval/
    eval_ablation.py
    eval_jamming.py
    eval_metrics.py
tests/
    test_channel_shapes.py
    test_power_constraint.py
    test_jammer_energy.py
    test_soft_codebook.py
scripts/
    run_ablation.sh
    run_jamming_sweep.sh
notebooks/
    01_channel_validation.ipynb
    02_layer_ablation.ipynb
    03_jamming_results.ipynb

```

4.3 Codex 입력 프롬프트 예시

아래 프롬프트를 VS Code 터미널에서 Codex CLI에 순차적으로 입력한다.

Prompt 1: 프로젝트 스캐폴드 생성

Create a PyTorch research project for speech-to-speech JSCC under Rayleigh fading and jamming. Do not implement digital FEC or bit-level RVQ index transmission. The project must use continuous or soft codec representations as the JSCC source. Create the directory structure, config files, and minimal runnable training/evaluation scripts. Add pytest tests for tensor shapes and power normalization.

Prompt 2: Rayleigh channel and jammer module

Implement `src/channels/rayleigh.py` and `src/channels/jammer.py`. Use complex PyTorch tensors. The Rayleigh channel should support flat fading and optional OFDM resource grids. The jammer module should support barrage, narrowband, burst, and pilot jamming masks. Include functions to compute effective SINR, JSR, and jammer mask statistics. Add unit tests that verify signal power, jammer power, and output tensor shapes.

Prompt 3: JSCC encoder/decoder

Implement `src/models/jscc_encoder.py` and `jscc_decoder.py`. The encoder maps codec latent tensors of shape $[B, L, T, D]$ plus channel-state vector c to complex channel symbols $[B, M]$ or $[B, K, N]$. Enforce average power normalization. The decoder maps received complex symbols plus channel-state vector to reconstructed latent tensors $[B, L, T, D]$. Include layer gating and optional layer-wise power allocation, but keep the first version simple and deterministic.

Prompt 4: Soft codebook projection

Implement `src/models/soft_codebook.py`. It should support two modes: (1) continuous latent reconstruction loss, and (2) soft codebook projection from token logits to codebook embeddings using temperature-scaled softmax. Include top-k token accuracy computation for analysis only. Do not convert the communication system into a digital index transmission pipeline.

Prompt 5: Codec wrapper

Create `src/codecs/base_codec.py` with an abstract interface: `encode_waveform`, `decode_representation`, `get_codebook`, `representation_shape`. Then create a placeholder `SpeechTokenizerWrapper` and `EnCodecWrapper`. If external pretrained models are unavailable, implement a mock codec that produces deterministic random latent tensors and reconstructs a dummy waveform so the JSCC pipeline can be tested end-to-end.

Prompt 6: Training loop

Implement `train_latent_jscc.py`. It should load codec representations, sample random SNR/JSR/jammer types per batch, pass through JSCC encoder, Rayleigh+jamming channel, JSCC decoder, and compute layer-weighted latent MSE plus power penalty. Log loss, effective SINR, JSR, layer-wise MSE, and reconstruction examples. Add config options for layer weights, SNR range, JSR range, and jammer probabilities.

Prompt 7: Evaluation sweep

Implement `eval_jamming.py` to sweep SNR, JSR, jammer type, and layer adaptation modes. Compare: uniform JSCC, rule-based channel-adaptive layer gating, and learned gating if available. Save CSV results and plots for layer-wise MSE, waveform metrics placeholders, and effective SINR. Keep ASR/WER only as optional evaluation, not part of the communication pipeline.

Prompt 8: Documentation

Write `README.md` explaining that this project studies speech-to-speech JSCC using neural speech codec representations under Rayleigh fading and jamming. Clearly state that the system does not use ASR/TTS as the main communication path, does not transmit model weights, and does not implement digital FEC-based RVQ index transmission. Include quickstart commands, config descriptions, and experiment commands.

참고자료

- X. Zhang et al., “SpeechTokenizer: Unified Speech Tokenizer for Speech Language Models”, arXiv:2308.16692.
- E. Bourtsoulatz, D. B. Kurka, D. Gündüz, “Deep Joint Source-Channel Coding for Wireless Image Transmission”, arXiv:1809.01733.
- Kyutai Labs, Moshi and Mimi documentation/repository. Mimi is a streaming neural audio codec used in Moshi.
- OpenAI Developers, Codex CLI documentation.